Instruktioner för MA1

Det här dokumentet innehåller material för den första modulen (MA1) i kursen Programmeringsteknik II. Vi tar upp rekursiva algoritmer och algoritmanalys.

> Övningarna som hör till modulen finns i den här sortens gula rutor i texten. Det finns **12 obligatoriska övningar** totalt i modulen. Efter teori och ett antal exempel följer en grupp övningar på samma tema. Läs instruktionerna nedan noga.

Modulen innehåller följande filer:

1. `MA1.py` : Detta är huvudfilen där dina övningslösningar ska finnas när du presenterar dem. Den ska också finnas med i zip-filen du laddar upp till Studium.
2. För var och en av övningarna finns det en tillhörande fil `MA1_test_`*namn*`.py` där *namn* är namnet på den funktion som testas (till exempel `MA1_test_largest.py` och `MA1_test_multiply.py`). Testerna använder ramverket unittest. Du behöver inte läsa på om hur hela det ramverket fungerar. Det räcker med att titta på själva testfallen.
3. Filerna `tests.sh` respektive `tests.bat` kan användas för att köra alla test från terminalen på olika operativsystem.

Följ dessa instruktioner när du förbereder dig för att presentera dina lösningar:

1. Uppgifterna ska lösas i den ordning de finns i texten. Du ska använda de verktyg (klasser, funktioner, metoder, ...) som tas upp i övningen. Syftet är inte bara att få rätt resultat, utan att öva specifika tekniker.
2. Om inget annat anges ska du inte använda några ytterligare paket eller färdig kod, utöver det som finns i de filer du har laddat ned med uppgiften.
3. Alla funktioner måste namnges, ha argument och returvärden på precis det sätt som anges i uppgiften.
4. Du kan skriva kod i `main`-funktionen som testar din implementation. Om du under din testning har skrivit någon kod på toppnivå (utanför alla funktioner) ska du ta bort den innan du skickar in. Detsamma gäller eventuella extra testutskrifter eller liknande.
5. Uppgifter som ska besvaras med text skrivs som kommentarer eller textsträngar sist i koddokumentet. Textsvaren kan hållas korta.
6. Uppgifterna ska först redovisas muntligt för assistent eller lärare och sedan laddas upp i Studium.
7. Under den muntliga redovisningen kan du först bli ombedd att legitimera dig. När presentationen är klar ska du fylla i de första raderna i dokumentet (med ditt namn, e-post, namn på granskande assistent/lärare, datum).

   När du är helt klar skapar du en zip-fil med filen `MA1.py` och dina testfunktioner. Ladda upp den filen till Studium.

   Om du har gjort figurer, vilket *kan* vara relevant i vissa av uppgifterna, ska de vara

som pdf, jpg eller png och finnas med i zip-filen.

Vi kommer inte att godta några andra format (Word, Jupyter Notebook, ...). Vi godtar heller inte inlämning via e-post.

Viktigt!
Du får samarbeta med andra studenter, söka på nätet och orientera dig med AI-verktyg, men du måste i slutänden skriva och kunna förklara din egen kod. Du får inte kopiera eller skriva av kod vare sig från andra studenter, automatiserade verktyg, eller från nätet förutom från de ställen som uttryckligen pekas ut i denna lektion.
Att byta variabelnamn och liknande ytliga redigeringar räknas inte som att skriva sin egen kod. Att hjälpa någon annan att fuska (om du exempelvis delar med dig av din kod till någon som försöker lämna in den som sin egen) är inte tillåtet och kommer att anmälas.
Eftersom uppgifterna ingår som moment i examinationen är vi skyldiga att anmäla markanta brott mot dessa regler till univeristets disciplinnämnd.
Allt kursmaterial skyddas av upphovsrätten och ska inte spridas vidare.

1 Algoritmkonstruktion med rekursion

Rekursion är ett kraftfullt medel för att finna effektiva algoritmer men det är också ett sätt att konstruera hopplöst ineffektiva algoritmer. Det är därför viktigt att man kan analysera algoritmen för att förstå om den är bra eller dålig.

Att lösa ett problem rekursivt innebär att man uttrycker lösningen i termer av samma typ av problem som dock måste vara i någon mening enklare. Man delar alltså upp problemet i ett eller flera delproblem av samma typ, löser dessa på samma sätt och kombinerar sedan lösningarna av delproblemen till en lösning av ursprungsproblemet.

Vi skall demonstrera hur det går till genom ett antal exempel. I många av exemplen finns det enkla iterativa algoritmer eller inbyggda funktioner/metoder som kan lösa problemen men vårt fokus är här att *öva rekursivt tänkande*!

Exempel 1: Fakultetsberäkning

Antag att vi vill skriva en funktion som beräknar $n! = n(n-1)(n-2)\cdot\ldots\cdot 2\cdot 1$ för ett naturligt tal $n \geq 1$. Man definierar också $0! = 1$.

En Python-funktion för att utföra beräkningen:

```
1 def fac(n):
2     p=1
3     for x in range(1,n+1):
4         p=p*x
5     return p
```

Denna algoritm som ansluter väl till den givna definitionen, sägs vara *iterativ* eftersom den använder en *iteration* (en `for`-sats) för att utföra beräkningen.

Det går också att definiera fakultetsuttrycket *rekursivt*:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n > 0 \end{cases}$$

Definitionen av $n!$ är rekursiv eftersom den använder sig av $(n-1)!$ dvs. av samma sak som ska definieras, ett steg närmare det fördefinierade basfallet $0! = 1$.

Detta kan utnyttjas i programmeringen genom att skriva en *rekursiv* funktion:

```
1 def fac(n):
2     if n==0:
3         return 1
4     else:
5         return n*fac(n-1)
```

Den är rekursiv eftersom den innehåller ett anrop till sig själv, dock med ett annat argument (`n-1` i stället för `n`) förutom om funktionen anropas med argumentet 0 då den returnerar direkt — ett så kallat *basfall*.

För att, till exempel, beräkna `fac(3)` måste först `fac(2)` beräknas. För att beräkna `fac(2)` måste `fac(1)` beräknas vilket i sin tur kräver att `fac(0)` beräknas. Värdet på `fac(0)` kräver inte något nytt anrop utan värdet kan returneras direkt.
Detta kan illustreras i vidstående figur:

```
fac(3):
|   3*fac(2)
|       |   2*fac(1)
|       |       |  1*fac(0)
|       |       |     1
|       |       1
|       2
6
```

När programmet kommer in i `fac(0)` finns det alltså ytterligare tre oavslutade anrop till `fac` var och en med sitt argument. Eftersom `fac(0)` inte anropar vidare kan den returnera sitt värde till den väntande `fac(1)`.

Den första multiplikationen som utförs är den innersta, på rad 4, och den sista är den på

rad 2.

Observera att det alltid måste finnas minst ett *basfall* som bryter anropskedjan (rekursionen).

Exempel 2: Beräkning av x^n – första version

Vi skall beräkna x^n, n heltal ≥ 0, med upprepade multiplikationer.

Iterativ definition: $x^n = x \cdot x \cdot x \cdot x \cdot \ldots \cdot x$

Iterativ funktion:

```
1 def power(x,n):
2     p=1
3     for i in range(1,n+1):
4         p=p*x
5     return p
```

Rekursiv definition:

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot x^{n-1} & \text{if } n > 0 \end{cases}$$

Rekursiv funktion:

```
1 def power(x, n):
2     if n == 0:
3         return 1
4     else:
5         return x*power(x, n-1)
```

Inklusive negativa n:

```
1 def power(x, n):
2     if n == 0:
3         return 1
4     elif n > 0:
5         return x*power(x, n-1)
6     else:
7         return 1./power(x, -n)
```

I avsnittet om algoritmanalys presenterar vi ett betydligt effektivare sätt att göra beräkningen.

Övning 1: Funktionen `multiply`

Skriv en rekursiv funktion (dvs. utan iteration) `multiply(m, n)` där `m` och `n` är två icke-negativa heltal. Funktionen ska returnera produkten `m*n` och beräkningen ska göras *utan* att använda multiplikation. Produkten skall alltså beräknas med *additioner*.

Om du skulle vilja minimera antalet rekursiva anrop hur skulle du då modifiera koden?

Övning 2: Harmoniska summan

Skriv en rekursiv funktion `harmonic(n)` som beräknar den harmoniska summan $h(n)$ definierad som

$$h(n) = 1 + 1/2 + 1/3 + \cdots + 1/n$$

för ett positivt heltal `n`.

Övning 3: Binär representation av heltal

Skriv en rekursiv funktion `get_binary(x)` som returnerar den binära representationen av heltalet `x` som en sträng.

Example:

`get_binary(0)` ska returnera '0'
`get_binary(1)` ska returnera '1'
`get_binary(5)` ska returnera '101'
`get_binary(15)` ska returnera '1111'
`get_binary(-18)` ska returnera '-10010'

Tips! Utgå från *två* basfall, för $x == 0$ och $x == 1$! Tänk på hur man skulle kunna uttrycka att skriva den binära representationen för ett negativt heltal tal i termer av att skriva den binära representationen av ett annat, positivt, heltal, respektive hur det går att uttrycka att skriva representationen av ett stort heltal i termer av att utöka representationen av ett specifikt, mindre, heltal (en siffra i taget).

2 Effektiva rekursiva algoritmer

I ovanstående exempel blir de rekursiva metoderna varken enklare eller effektivare än den iterativa men rekursiva resonemang är ett ändå kraftfullt sätt att lösa problem och hitta effektiva algoritmer.

Att uttrycka sig rekursivt är ofta naturligt i matematiken. Deriveringsreglerna ("derivatan av en summa är summan av derivatorna") är ett exempel på detta. Inte desto mindre brukar rekursion betraktas som svårt när man börjar med det i programmering. Orsaken till detta är säkert att man i de flesta fall lärt sig programmera med iterationer.

Frågor att besvara vid konstruktion av en rekursiv algoritm:

1. Hur kan jag dela upp ursprungsproblemet i mindre problem av samma slag?
2. Hur kombinerar jag lösningarna till delproblemen till en lösning på ursprungsproblemet?
3. Vilka rekursionsterminerande fall är lämpliga? Kommer man alltid till ett sådant oberoende av indata?

Detta är mycket besläktat med induktionsbevis. Vi antar i det fallet att en sats gäller för ett antal fall och visar hur ett större fall måste gälla om det gäller för specifika mindre fall, ofta att om satsen gäller för $n - 1$ måste den även gälla för n.

För rekursiva algoritmar antar vi att vi kan lösa problemet för ett mindre problem och utgår från den lösningen för att även hantera ett större problem. Precis som i induktionsbevis så

måste man ha minst ett basfall som bevisas/implementeras separat.

Exempel 3: Skapa en omvänd sträng

Antag att vi vill skriva en funktion som skriver ut en sträng där tecknen kommer i omvänd ordning.

Hur definierar man problemet i termer av sig självt? Vad är basfallet?

Induktionsantagande: Antag att vi kan lösa problemet för de $n-1$ första tecknen i strängen.

Basfall: Att skriva ut en sträng med *ett* tecken i omvänd ordning. Trivialt.

Vi kan omsätta detta i kod:

```
1  def reverse_string(s):
2    if len(s)<=1:
3        print(s, end='')
4    else:
5        print(s[-1], end='')      # Print the last character
6        reverse_string(s[:-1])  # All but the last character in reverse order
```

Exempel 4: Stapla brickor

En mängd med n brickor av olika storlek ligger travade på en plats (f) i storleksordning med den största underst. Problemet går ut på att flytta hela traven till en annan plats (t) under iakttagande av följande regler:

1. Endast en bricka får flyttas per gång.
2. En större bricka får aldrig läggas på en mindre.

Till hjälp har man ytterligare en plats (h) som får användas för mellanlagring.

Då kan vi utföra operationen på följande sätt:

Induktionsantagande: Vi kan lösa problemet med $n-1$ brickor.

Basfall: Flytta 0 brickor. Trivialt – gör ingenting!

Induktionssteg:

1. Flytta de $n-1$ översta brickorna till h, med t som mellanlagring.
2. Flytta den kvarvarande från f till t.
3. Flytta de $n-1$ brickorna på h till t, med f som mellanlagring.

Problemet löses således rekursivt genom att lösa två problem av storlek $n-1$.

Vi kan definiera vårt basfall som antingen $n=1$ eller $n=0$. ($n=0$ leder till enklare kod

i den tillhörande övningen!)

Övning 4: Alternativ `reverse_string`.

Skriv en rekursiv funktion `reverse_string(s)` som *returnerar en sträng* (alltså inte skriver ut den) där tecknen kommer i omvänd ordning mot tecknen i `s`.

Krav: Använd antagandet att vi kan lösa problemet för de $n-1$ *sista* tecknen i strängen.

Obs! Det är mycket bättre att returnera ett värde än att skriva ut det på skärmen. Det är den som anropar funktionen som vet på vilket sätt värdet behövs, så det bör vara anroparens ansvar att skriva ut det, om det är vad som önskas.

Man behöver inte dela vid första eller sista tecknet — det går bra att dela strängen var som helst, t.ex. i mitten:

```
1 def reverse_string(x):
2     if len(x) <= 1:
3         return x
4     else:
5         mid = len(x)//2
6         return reverse_string(x[mid:]) +
          ↪  reverse_string(x[:mid])
```

På rad 6 ser vi att funktionen anropar sig själv två gånger. Man säger då att programmet är *trädrekursivt*. I detta fall spelar det ingen större roll vilken uppdelning vi väljer men i diskussionen om algoritmanalys kommer vi se att valet av delproblem kan vara avgörande för algoritmens effektivitet.

Övning 5: Funktionen `largest`

Skriv en rekursiv funktion `largest(a)` som returnerar det största värdet av elementen i listan `a`. Du kan anta att listan inte är tom och att den bara innehåller jämförbara element (t.ex. alla tal eller alla strängar).

Krav: Förändra inte listan i argumentet. Du får inte använda funktionen `max` för att lösa uppgiften. Lösningen måste ha en bättre komplexitet än exponentiell.

Övning 6: Funktionen `count`

Skriv en rekursiv funktion `count(x, s)` som räknar och returnerar hur många gånger `x` finns på högsta nivån i listan `s`, inklusive inre listor i `s`.
Exempel: Anropet `count(4, [1, 4, 2, ['a', [[4], 3, 4]]])` ska returnera 1.
Modifiera sedan koden så att den räknar förekomster på *alla* nivåer. Anropet ovan ska då returnera 3.
Tips! Uttrycket `type(x) == list` returnerar `True` om x är en lista, annars `False`.
Requirements: Make sure that lists are not destroyed. Searching of non-existing elements should return 0.
The following points should be checked:

1. Sökning i en tom lista ska returnera 0.
2. Tester behövs för fallen då det eftersökta elementet finns först eller sist.
3. Det ska gå att söka efter listor (`x` kan i sig vara en lista).
4. Det ska gå att söka genom flera nivåer av sublistor.
5. Sökning efter element som inte finns ska returnera 0.
6. Listan som genomsöks får inte förändras på något sätt.

Övning 7: Brickleken

Skriv en funktion `def bricklek (f, t, h, n)` som returnerar en lista med instruktioner om hur brickorna ska flyttas. Parametrarna `f`, `t` och `h` är strängar som identifierar de olika platserna och `n` är antalet brickor.
Anropet `bricklek('f', 't', 'h', 2)` ska returnera listan `['f->h', 'f->t', 'h->t']` och ska tolkas som "*flytta från `f` till `h`, flytta från `f` till `t`, flytta från `h` till `t`*".
Varje element i listan är alltså en sträng som anger varifrån och vart. Eftersom det alltid är den översta brickan som ska flyttas behövs ingen annan information. Det är viktigt att strängarna ser ut exakt som i exemplet och inte innehåller några blanktecken!
Tips: Funktionen behöver inte mer än 5 rader, inklusive `def`-raden!

Rekursionsdjup

I vissa av ovanstående och kommande uppgifter och exempel kan man få problem med det så kallade *rekursionsdjupet*. `largest` är ett av de fall där det kan uppstå. Problemet handlar om att det minne som är reserverat för att hantera funktionsanrop tar slut. Generellt sett kan man säga att rekursion över mycket långa listor kan vara problematiskt. Vi poängterar igen att många av uppgifterna ovan är till för att öva rekursivt tänkande.

3 Algoritmanalys

I algoritmanalysen studerar man hur tiden för en algoritm beror av problemstorleken. Av tradition kallar man detta för algoritmens *komplexitet* vilket alltså är ett uttryck för algoritmens effektivitet som inte har något att göra med hur svår algoritmen är att förstå eller implementera.

De resultat analysen ger blir av slaget "körtiden för den här algoritmen växer proportionellt med kvadraten på problemstorleken".

I ställlet för att mäta tid kan vi ofta välja att prata om hur många gånger en viss typ av operation utförs.

Exempel 5: Beräkning av x^n igen

I avsnittet om rekursion presenterade vi två olika sätt att beräkna x^n.

Det första gjordes iterativt med en for-sats:

```
1 def power(x,n):
2     p=1
3     for i in range(n):
4         p=p*x
5     return p
```

Denna funktion utför uppenbarligen n multiplikationer. Om x är ett vanligt tal så tar alla multiplikationer lika lång tid, oavsett värdet på x. Det betyder att vi kan räkna med att tiden för denna funktion växer linjärt med n. Man brukar då säga att *komplexiteten* är $\mathcal{O}(n)$ (uttalas "Ordo" eller, på engelska "Big Oh") eller $\Theta(n)$ (uttalas "Theta"). Vi återkommer längre fram till de exakta definitionerna av dessa termer.

I det andra sättet skrev vi en rekursiv funktion:

```
1 def power(x, n):
2     if n == 0:
3         return 1
4     else:
5         return x*power(x, n-1)
```

Den här funktionen kommer att göra n funktionsanrop och n multiplikationer så även denna har komplexiteten $\Theta(n)$.

En annan möjlig definition är:

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ (x^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ x \cdot (x^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases} \tag{1}$$

Även denna definition är rekursiv, men i stället för att återföra lösningen av ett problem av storleken n på ett problem av storleken $n-1$, så använder vi oss av problem av storleken $n/2$, dvs. ett betydligt mindre problem.

```
1  def power(x, n):
2      if (n == 0):
3          return 1.0
4      else:
5          p = power(x, n//2)  # Integer division
6          if (n % 2 == 0):    # n even
7              return  p*p
8          else:               # n odd
9              return  x*p*p
10
```

Anropet `power(x,1000)` kommer att generera följande sekvens av anrop:

```
power(x,1000) -> power(x,500) -> power(x,250) -> power(x,125) ->
power(x,62)   -> power(x,31)  -> power(x,15)  -> power(x,7)   ->
power(x,3)    -> power(x,1)   -> power(x,0)
```

Här kommer alltså resultatet att beräknas med 10 funktionsanrop och 15 multiplikationer (10 kvadreringar och 5 extra multiplikationer när argumentet är udda). Detta är ju en betydande förbättring över de två första implementationerna.

Eftersom problemet halveras varje rekursionssteg så blir det generellt ungefär $\log_2(n)$ anrop (i själva verket $\lfloor \log_2(n) \rfloor + 1$ men vi gör det lite enklare för oss) och lika många kvadreringar plus högst lika många extra multiplikationer för att hantera udda argument. Tiden bör alltså växa proportionellt mot $\log(n)$. Man säger då att algoritmen har komplexiteten $\Theta(\log(n))$

Asymptotisk notation

Man använder Ordo- och Theta-uttrycken för att enkelt beskriva den *väsentliga* tillväxten utan att behöva räkna ut exakt hur uttrycken ser ut.

En (matematisk) funktion $t(n)$ är $\mathcal{O}(f(n))$ om det existerar två konstanter c och n_0 sådana att:

$$|t(n)| \leq c \cdot |f(n)| \qquad \forall n > n_0 \tag{2}$$

Om vi applicerar denna definition på exemplen med antalet multiplikationer ovan, betecknar $|t(n)|$ antalet multiplikationer som behöver utföras. Använder vi någon av de två första definitionerna är $c = 1$ och $f(n) = n$. Med definition (1) är $f(n) = \log_2(n)$ och $c = 2$. I samtliga fall är $n_0 = 0$.

I de här exemplen används alltså $\mathcal{O}$ för att ange (en övre gräns för) hur många gånger en central operation (i det här fallet multiplikation) används. Notationen kan också användas för att ange en övre gräns för hur lång tid eller hur stort lagringsutrymme en algoritm behöver. Tidsåtgången för en algoritm växer normalt lika fort som antalet operationer, men värdet på c beror då även på sådant som vilket språk algoritmen är implementerad i och vilken dator implementationen körs på.

Eftersom $\mathcal{O}$ betecknar en övre gräns, är en algoritm som är $\mathcal{O}(\log(n))$ också $\mathcal{O}(n)$, $\mathcal{O}(n^2)$, $\mathcal{O}(2^n)$ och så vidare.

Tecknet Ω används för att ange en undre gräns: En funktion $t(n)$ är $\Omega(f(n))$ om det existerar två konstanter c och n_0 sådana att:

$$|t(n)| \geq c \cdot |f(n)| \qquad \forall n > n_0 \tag{3}$$

Om funktionen $t(n)$ är både $\mathcal{O}(f(n))$ och $\Omega(f(n))$ sägs den vara $\Theta(f(n))$. (Naturligtvis måste man använda olika konstanter c för den övre och den undre gränsen.)

Ofta säger man $\mathcal{O}$ när man egentligen menar Θ.

Notera att det inte spelar någon roll vilken logaritm man använder i $\mathcal{O}$-, Θ- och Ω-uttrycken, eftersom

$$\log_a(x) = \frac{1}{\log_b(a)} \cdot \log_b(x)$$

och $\frac{1}{\log_b(a)}$ kan bakas in i c. Därför behöver man heller inte ange vilken logaritm som avses.

Exempel 6: Indexering i Python-listor

Antag att du har en lista som du ska hämta ett element från en given position (första, nittonde, sista, ...). Detta är ett exempel på när tidsåtgången inte växer med problemstorleken. Oavsett hur lång listan är kommer det att krävas lika många operationer för att hämta elementet. Indexering i listor är alltså $\Theta(1)$ eller $\mathcal{O}(1)$ vilket är samma sak.

Observera att andra programmeringsspråk kan ha andra sätt att implementera listor som då kan ge att indexering kräver $\Theta(n)$ operationer.

Exempel 7: Sökning i osorterad lista

När det gäller sökning brukar man vid algoritmanalys skilja på *lyckad* och *misslyckad* sökning.

Om värdet vi letar efter inte finns i listan, säger vi att sökningen misslyckas. En misslyckad sökning i en osorterad lista kräver att vi kontrollerar varje element i listan. (Hur ska vi annars kunna vara säkra på att värdet inte finns?) Antalet operationer växer alltså linjärt med problemstorleken (dvs. listans längd) och vi säger att en misslyckad sökning i en osorterad lista är $\Theta(n)$.

Om det eftersökta värdet finns i listan, säger vi att sökningen lyckas. Här är det relevant att skilja på bästa fall, värsta fall och genomsnitt. I *värsta fall* behöver vi leta igenom hela listan innan vi hittar det eftersökta värdet. Under förutsättning att alla värden är lika sannolika att bli sökta behöver vi i *genomsnitt* leta igenom halva listan för att hitta det eftersökta värdet. I båda dessa fall växer antalet operationer linjärt med problemstorleken. I *bästa fall* hittar vi det eftersökta värdet på det första stället vi letar på. Då behövs ett konstant antal operationer — det spelar ingen roll hur lång listan är. En lyckad sökning i en osorterad lista är alltså:

- $\Theta(1)$ bästa fall,
- $\Theta(n)$ värsta fall,
- $\Theta(n)$ i genomsnitt.

Generellt är man oftast intresserad av en algoritms komplexitet i värsta fall och/eller i genomsnitt.

Observera att koden på rad 7 respektive 10 i vidstående exempel båda har komplexiteten $\Theta(n)$ (där n är listans längd) men den på rad 10 är både snabbare och elegantare!

Att en operation är inbyggd i språket ändrar inte på komplexiteten även om den ofta väsentligt påverkar tiden för operationen.

```
1  def search(x, lst):
2      for i in range(len(lst)):
3          if x == lst[i]:
4              return True
5      return False
6
7  if search(x, mylist):
8      do_something()
9
10 if x in mylist:
11     do_something()
12
```

Exempel 8: Sökning i sorterad lista

Om listan som vi ska söka i är sorterad i storleksordning, minskar mängden arbete avsevärt. I detta exempel kallar vi listan för `a` och det eftersökta värdet för `v`. Vi antar att elementen i `a` är sorterade i stigande ordning, dvs. så att `a[i+1]` $\geq$ `a[i]` för alla index `i`.

En sökning går då till så här:

1. Om listan är tom har sökningen misslyckats. `v` kan ju omöjligt finnas i en tom lista.
2. Om `a[n//2] == v` är vi klara.
3. Annars, upprepa proceduren, den här gången med en hälften så lång lista:
   - Om `a[n//2] < v`, ge `a[n//2+1 : n]` (dvs. den delen av `a` som ligger till höger om mittelementet) som input.
   - Om `a[n//2] > v`, ge `a[0 : n//2]` (dvs. den delen av `a` som ligger till vänster om mittelementet) som input.

På det här sättet halverar vi problemstorleken i varje steg. Jämför med hur du själv skulle leta i en mängd sorterade element, exempelvis en telefonkatalog, en klasslista eller registret i en bok!

Vid en misslyckad sökning behöver vi fortsätta tills listan är tom. Om vi skulle dubblera `a`:s längd, skulle antalet nödvändiga operationer öka med 1. Med andra ord växer mängden arbete logaritmiskt; algoritmen är $\Theta(\log(n))$.

Vid en lyckad sökning behöver vi utföra samma procedur, men kommer i genomsnitt att hitta `v` efter halva tiden. Mängden arbete växer alltså som $0,5 \cdot \log(n)$, men $0,5$ är bara en konstant och kan bakas in i c. Algoritmen är alltså $\Theta(\log(n))$ i genomsnitt.

Frågan blir hur många halveringar som behövs för längden n:

$$\frac{n}{2^k} = 1$$

vilket ger $k = \lfloor \log_2(n) \rfloor$

Exempel 9: Hash tables, dictionaries

Genom att sortera posterna i storleksordning går det alltså att minska komplexiteten för sökning från $\Theta(n)$ till $\Theta(\log n)$ vilket är en dramatisk förbättring för stora mängder. Det går dock att göra det ännu bättre genom så kallade *hashtabeller*. Python använder denna teknik för att implementera lexikon (”dictionaries”). I sådana är tiden för sökoperationen oberoende av storleken på mängden, vilket man skriver $\Theta(1)$ (eller $\mathcal{O}(1)$ vilket är samma sak).

Vi återkommer till hashtabeller i ett senare avsnitt i kursen som behandlar datastrukturer.

Exempel 10: Urvalssortering

Bredvidstående kod implementerar en enkel sorteringsalgoritm som kallas *urvalssortering*.
(Observera den "pythonska" konstruktionen på rad 7 för att byta innehåll på två variabler!)

```
1 def selection_sort(a):
2     for i in range(len(a)-1):
3         k = i
4         for j in range(i+1, len(a)):
5             if a[j] < a[k]:
6                 k = j
7         a[i], a[k] = a[k], a[i]
8
```

Om vi betecknar längden av `a` med n så ser vi att den första `for`-satsen exekverar $n-1$ gånger med i-värdena $0, 1, 2, \ldots, n-2$. Den inre loopen exekverar första gången $n-1$ gånger, andra gången $n-2$ gånger etc. Jämförelsen på rad 5 kommer således utföras

$$(n-1) + (n-2) + (n-3) + \cdots + 2 + 1 = \frac{n(n-1)}{2}$$

som är $\Theta(n^2)$ (övning: bevisa det!).

Tiden för koden på raderna 5–6 betraktar vi som konstant vilket gör detta till en typisk $\Theta(n^2)$-metod.

Exempel 11: Analys av brickleken

Här löser vi ett problem av storleken n med hjälp av två problem av storleken $n-1$ plus en brickförflyttning. Om vi låter $t(n)$ stå för antalet brickförflyttningar så gäller alltså

$$t(n) = \begin{cases} 1 & \text{if } n = 1 \\ 1 + 2 \cdot t(n-1) & \text{if } n > 1 \end{cases}$$

Den här differensekvationen kan enkelt lösas genom "teleskopering" dvs. att vi applicerar ekvationen om och om igen:

$$\begin{aligned} t(n) &= 1 + 2 \cdot t(n-1) = 1 + 2 \cdot [1 + 2 \cdot t(n-2)] \\ &= 1 + 2 + 4 \cdot t(n-2) = 1 + 2 + 4 \cdot [1 + 2 \cdot t(n-3)] \ldots \\ &= 1 + 2 + 4 + \ldots + 2^k \cdot t(n-k) \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

Exempel 12: Fibonaccitalen

Fibonaccitalen är en följd av tal som börjar med 0 och 1 och därefter är varje tal summan av de två närmast föregående talen: 0, 1, 1, 2, 3, 5, 8, 13, 21 ...

Så här kan en funktion för att beräkna det n:te talet (vi numrerar dem med början på 0) se ut:

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

Denna algoritm är lite svårare att analysera.

I stället för att titta på *tid* så undersöker vi hur *många additioner* anropet `fib(n)` ger upphov till. Om n är 0 eller 1 görs inga additioner. Om n är större än 1 så görs 1 addition *plus* antalet additioner som görs av anropen `fib(n-1)` och `fib(n-2)`.

Om vi låter $t(n)$ beteckna antalet additioner som utförs av anropet `fib(n)` så gäller alltså:

$$t(n) = \begin{cases} 0 & \text{if } n \leq 1, \\ 1 + t(n-1) + t(n-2) & \text{if } n > 1. \end{cases} \tag{4}$$

Observera likheten med Fibonaccitalen!

Detta är en linjär differensekvation. Vi ger lösningen nedan men vi börjar med att göra en enklare uppskattning.

Eftersom $t(n)$ måste vara en växande funktion så kan vi göra en uppskattning uppåt:

$$t(n) = 1 + t(n-1) + t(n-2) < 1 + 2 \cdot t(n-1)$$

Vi löste detta som en *ekvation* vid analysen av brickleken ovan och konstaterade att lösningen till denna var $2^n - 1$. Eftersom detta är en *överskattning* kan vi konstatera att antalet additioner (och därmed även tiden) är $\mathcal{O}(2^n)$.

Vi skulle kunna genomföra en uppskattning nedåt på ett liknande sätt:

$$t(n) = 1 + t(n-1) + t(n-2) > 1 + 2 \cdot t(n-2) \tag{5}$$

vilket ger oss att $t(n)$ är $\Omega((\sqrt{2})^n)$ och därmed att antalet additioner växer snabbare än $(1.4142)^n$.

Differensekvationen (4) går att lösa exakt och ger $t(n) \approx 1.618^n$. Vi ger beviset här om du är nyfiken, men härledningar av det här slaget är utanför det ordinarie kursinnehållet.

Proof. Den karaktäristiska ekvationen för den homogena ekvationen är

$$r^2 - r - 1 = 0,$$

som har lösningen

$$r_{1,2} = \frac{1 \pm \sqrt{5}}{2},$$

dvs. den homogena ekvationen har lösningen

$$F(n) = ar_1^n + br_2^n,$$

där a och b bestäms ur begynnelsevillkoren. Eftersom

$$t(n) = -1$$

är en partikulärlösning, kan den allmänna lösningen skrivas

$$t(n) = ar_1^n + br_2^n - 1.$$

Med hjälp av begynnelsevillkoren kan a och b bestämmas och ger lösningen

$$\begin{aligned} a =& (1 - r_2)/(r_1 - r_2) \\ b =& -(1 - r_1)/(r_1 - r_2) \end{aligned}$$

Eftersom $r_1 \approx 1.618$ och $r_2 \approx 0.618 < 1$ så ser man att, för stora n, gäller

$$t(n) \approx 1.618^n$$

för stora n.

Antalet additioner växer således exponentiellt och därmed växer också tiden $t(n)$ exponentiellt.

□

Programmet är *trädrekursivt*, det vill säga varje anrop resulterar i två nya anrop. Detta kan potentiellt ge orimliga exekveringstider.

Problemet med koden är att vi kommer att lösa samma problem många gånger (av utrymmesskäl skriver vi f i stället för `fib`):

$$\begin{aligned} f(5) &= f(4) + f(3) \\ &= (f(3) + f(2)) + (f(2) + f(1)) \\ &= ((f(2) + f(1)) + (f(1) + f(0)) + ((f(1) + f(0)) + 1) \\ &= (f(1) + f(0)) + 1 + 1 + 0 + 1 + 0 + 1 \\ &= 1 + 0 + 1 + 1 + 0 + 1 + 0 + 1 = 5 \end{aligned}$$

Vi ser alltså att f(4) anropas 1 gång, f(3) anropas 2 gånger, f(2) anropas 3 gånger, f(1) anropas 5 gånger och f(0) 3 gånger.

Det vanliga sättet att beräkna Fibonacci-tal är att i stället börja nerifrån och successivt beräkna f(0), f(1), ...f(n) vilket då görs med $\Theta(n)$ additioner.

Vi ska emellertid här passa på att introducera en annan teknik som kallas *memoisering* eller, ibland, *dynamisk programmering*. Problemet med den rekursiva lösningen är ju att man löser identiska problem många gånger. Memoisering innebär att man, när man löst ett visst delproblem, sparar denna lösning och, innan man ger sig på ett nytt delproblem,

först kontrollerar om det redan är löst. Om det är löst så använder man sig av lösningen annars löser man det enligt algoritmen.

För Fibonaccitalen karaktäriseras problemet entydigt av talets ”nummer”, alltså parametern n i koden. Vi kan då använda ett lexikon för att spara lösningarna:

```
memory = {0:0, 1:1}

def fib_mem(n):
    if n not in memory:
        memory[n] = fib_mem(n-1) + fib_mem(n-2)
    return memory[n]
```

Ett snyggare sätt

Funktionen `fib_mem` ovan använder en *global* variabel `memory` för att hålla reda på beräknade värden. Det vore snyggare om detta lexikon kunde döljas inuti funktionen. Man vill i allmänhet undvika globala variabler, särskilt om de som här bara används på ett enda ställe i programmet. Det fungerar dock inte att bara flytta in satsen `memory = {0:0, 1:1}`, för då kommer varje instans av funktionen skapa sin egen `memory` så redan beräknade värden kommer att ligga i *olika* lexikon (dock alla med namnet `memory`).

Följande kod löser däremot problemet:

```
def fib(n):
    memory = {0:0, 1:1}

    def _fib_mem(n):
        if n not in memory:
            memory[n] = _fib_mem(n-1) + _fib_mem(n-2)
        return memory[n]

    return _fib_mem(n)
```

Vi har återtagit namnet `fib` för den som ska användas. Inuti den skapar vi dels lexikonet `memory` och dels hjälpfunktionen `fib_mem`. Båda dessa är lokala i `fib` men alla instanser av `fib_mem` använder samma `memory`.

Denna väsentliga förbättring av den ursprungliga `fib` kan alltså göras helt ”osynlig” för den anropande koden.

Du kan se mer exempel på *omfång* (en. *scope*) för variabler i en YouTube-lektion från Corey Schafer.

En annan beskrivning av att använda memoisering för att beräkna Fibonacci-tal finns i en YouTube-lektion från Socratica.

Ett alternativt sätt att lösa problemet är att skicka `memory` som parameter:

```
def fib(n, memory = None):
    if memory is None:
        memory = {0: 0, 1: 1}
    if n not in memory:
        memory[n] = fib(n-1, memory) + fib(n-2, memory)
    return memory[n]
```

Om man anropar utan att ange `memory` kommer alltså lexikonet att skapas och sedan skickas med i de rekursiva anropen.

Tekniken att skriva funktioner lokalt i funktioner kommer vi att använda fler gånger i kursen.

Som vi ser i avsnittet om sortering nedan, behöver trädrekursiva algoritmer inte nödvändigtvis ge orimliga exekveringstider. I själva verket är många klassiska effektiva algoritmer (t.ex. sorteringsalgoritmer och snabb Fouriertransform) trädrekursiva.

Övning 8: Tidsåtgång för brickleken.

Antag att du har en hög med 50 brickor och det tar 1 sek att flytta en bricka. Hur lång tid kommer det att ta för dig att genomföra hela förflyttningen? Svara med lämpliga enheter!

Övning 9: Körtid för den givna Fibonacci-funktionen.

a) Verifiera genom testkörning att tiden för den givna algoritmen växer som $\Theta(1.618^n)$

b) Tag reda på hur lång tid anropen `fib(50)` respektive `fib(100)` tar (skulle ta) på din dator. Svara med lämpliga enheter! (Sekunder är inte en lämplig enhet om det tar flera timmar. Timmar är inte en lämplig enhet om det tar flera dagar eller år.)

Obs! Du ska inte försöka optimera koden, utan bara använda den givna funktionen!

Tips 1! Om du vill mäta tidsåtgången i ett Python-program kan du använda funktionen `perf_counter` i modulen `time`.

Exempel:

```
import time
tstart = time.perf_counter()
    # code to be timed
tstop = time.perf_counter()
print(f"Measured time: {tstop-tstart} seconds")
```

Tips 2! Du kommer inte kunna provköra `fib(100)`. Du **måste** uppskatta tiden genom att räkna!

Övning 10: `fib_mem(100)`

Vad är Fibonaccitalet nr 100 och hur lång tid tog det att beräkna det med `fib_mem`-metoden?

4 Sortering

Här presenterar och analyserar vi två rekursiva sorteringsalgoritmer.

Instickssortering

Den vanliga enkla instickssorteringen kan uttryckas rekursivt enligt följande:

Induktionsantagande: Vi vet hur man sorterar $n-1$ element.

Basfall: Vi kan sortera ett element.

Rekursionssteg:

Skjut in det n:te elementet på rätt plats bland de redan sorterade $n-1$ elementen, så att listan fortfarande är sorterad efter insticket.

```
def insertion_sort(l, n):
    if n > 1:
        insertion_sort( l, n-1 )    # sort the n-1 first
        x = l[n-1]
        i = n-2                     # move elements larger than x to the right
        while i>=0 and l[i]>x :
            l[i+1] = l[i]
            i=i-1
        l[i+1] = x                  # put in x
```

Titta på en visualisering på YouTube!

Analys

Tiden för att sortera n tal med denna algoritm beror på hur talen är permuterade.

I *bästa fall* är elementen redan sorterade. Det innebär att `while`-loopen aldrig exekverar. Det blir alltså sammanlagt $n-1$ anrop som tar konstant tid dvs. $\Theta(n)$.

I *värsta fall* är talen sorterade i omvänd ordning. Varje anrop kommer då också att innehålla en iteration som går $n-1$ gånger. d.v.s. totalt

$$(n-1)+(n-2)+\ldots+2+1=\frac{(n-1)n}{2}$$

gånger dvs. $\Theta(n^2)$

I *genomsnitt* kan man tänka sig att varje element ”reser” halva i stället för hela vägen som i värsta fallet. Summan blir då $(n-1)n/4$ vilket också är $\Theta(n^2)$.

Samsortering (”Merge sort”)

Nästa algoritm vi går igenom kallas för *mergesort* (eller *samsortering* på svenska). Denna algoritm är, som analysen och övningen nedan visar, väsentligt effektivare än instickssorteringen.

Induktionsantagande: Vi kan sortera $n/2$ element.

Induktionssteg: Dela mängden i två delar med vardera $n/2$ element, sortera dessa var för sig och sammanfoga sedan de två sorterade delarna.

Basfall: En mängd med 0 eller 1 element är sorterad.

Bredvidstående kod är avsedd att illustrera *idéen* med samsorteringen.

Det är *inte* en bra implementation för stora mängder data.

Ett problem är att hjälpmetoden `merge` rekurserar över listans längd vilket gör att ett arbetsutrymme (”stacken”) tar slut vid stora mängder data. Detta är ett allmänt problem med mycket djup rekursion.

```
1  def merge_sort(l):
2      if len(l) <= 1:
3          return l.copy()
4      else:
5          n = len(l)//2
6          l1 = merge_sort(l[:n])
7          l2 = merge_sort(l[n:])
8          return merge(l1, l2)
9
10 def merge(l1, l2):
11     if len(l1) == 0:
12         return l2
13     elif len(l2) == 0:
14         return l1
15     elif l1[0] <= l2[0]:
16         return [l1[0]] + merge(l1[1:], l2)
17     else:
18         return [l2[0]] + merge(l1, l2[1:])
19
20
21
```

Titta på en visualisering på YouTube!

Arbetet att sammanfoga de två sorterade delarna (hjälpfunktionen `merge` ovan) är proportionellt mot antalet element. (Uttrycket ”tiden är $\Theta(f(n))$” kan, löst uttryckt, tolkas som att tiden är proportionell mot $f(n)$ dvs. $t(n) = c \cdot f(n)$.)

Låt $t(n)$ beteckna tiden att sortera n element. Då gäller

$$t(n) = \begin{cases} c & \text{if } n = 0, \\ 2 \cdot t(n/2) + dn & \text{if } n > 0. \end{cases}$$

Om n är enexakt tvåpotens, d.v.s. $n = 2^k$ med ett heltal k gäller att

$$\begin{aligned} t(n) =& 2t(n/2) + dn = 2(2t(n/4) + dn/2) + dn \\ =& 4t(n/4) + dn + dn \ldots \\ =& 2^k t(n/2^k) + dnk \\ =& nt(1) + dn \log n \end{aligned}$$

Tiden är alltså $\Theta(n \log n)$.

Övning 11: Jämförelse av sorteringsmetoder

Antag att insticksortering och mergesort tar lika lång tid för 1000 slumptal – säg 1 sekund. Hur lång tid tar det för respektive algoritm att sortera 10^6 respektive 10^9 slumptal? Svara med lämpliga enheter!

Som vi har sett i ett par olika exempel ovan, kan tidsåtgången för algoritmer snabbt bli orimligt stor om man väljer fel algoritm (eller om det inte finns någon effektiv algoritm tillgänglig). Vi vill ha algoritmer för vilka tidsåtgången växer långsamt. Tiden för att lösa ett litet problem är ofta försumbar; det är när problemstorleken växer som det börjar bli viktigt att ha en effektiv algoritm!

Som följande övning visar ska man dock inte stirra sig blind på komplexiteten. I praktiken kan till exempel en algoritm som är $\Theta(n \cdot \log(n))$ vara minst lika effektiv som en algoritm som är $\Theta(n)$, om konstanten (c) är mycket större för den senare.

Övning 12: Teoretisk jämförelse av $\Theta(n)$ och $\Theta(n \cdot \log n)$

Antag att du kan välja mellan två algoritmer, A och B, för att lösa ett problem. Vi låter n beteckna antalet element i den datastruktur som algoritmerna opererar på. Du vet att algoritm A löser ett problem av storlek n på n sekunder. Tidsåtgången för algoritm B är $c \cdot n \cdot \log(n)$ sekunder, där c är en konstant. Du testkör algoritm B på din dator och finner att den tar 1 sekund på sig för att lösa ett problem när $n = 10$. Hur stort måste n vara för att algoritm A ska ta kortare tid på sig än algoritm B?

Lärdomar

Exponentiell tillväxt: Hopplöst att lösa stora problem!

Hur ska man veta om en rekursiv lösning är bra eller ej?

Om lösningen av ett problem är baserad på lösningar av två eller flera problem som är nästan lika stora som ursprungsproblemet så kommer man att få en exponentiell tillväxt vilket gör det omöjligt att klara stora problem. Koden för Fibonacci-talen liksom brickleken är exempel på sådana algoritmer. Som vi har sett kan memoisering förvandla en sådan lösning till att bli effektiv.

Om delproblemen är väsentligt mindre än ursprungsproblemet (t.ex. hälften så stora) kan det vara effektivt även om det finns flera delproblem på varje nivå. Kvadreringsalgoritmen för att beräkna x^n liksom samsorteringsalgoritmen är exempel på det.

I sorteringsexemplet visade det sig att basera lösningen på *två* delproblem av storlek $n/2$ var väsentligt bättre än att basera den på *ett* delproblem av storlek $n - 1$.

En annan observation är att ett $\log n$-beroende är *mycket* bättre än ett n-beroende men att $\log n$ inte är mycket sämre än konstant.